

July 3, 2026

```
[1]: # Numpy für bessere Berechnungen
import numpy as np
from numpy import exp, sqrt, log, pi
from uncertainties import unumpy as unp

# Weiteres für bessere Rechnungen
import pylab as py

from decimal import Decimal, ROUND_CEILING, ROUND_HALF_UP, getcontext #
    ↳ Besonders für sig. Runden
import math

# Berechnungen und Plotting
from scipy import odr
import scipy.optimize
import scipy.constants as c
from scipy.stats import norm
from scipy.optimize import curve_fit
from scipy.stats import chi2
from scipy.stats import poisson
from scipy.integrate import quad
from scipy.signal import find_peaks
from scipy.signal import argrelextrema, argrelmin, argrelmax
from scipy.special import factorial
import scipy.integrate as integrate

%matplotlib widget
import matplotlib.pyplot as plt
from matplotlib import colormaps
import matplotlib.mlab as mlab
import matplotlib.transforms as transforms

# Zum Auslesen von Dateien und ähnlichem
import os
import os.path
from pathlib import Path
```

```

import pandas as pd # Auch wichtig für den Latex Export
from pytexit import py2tex
import csv
import re
from typing import List

# Besseres Funktionen handling
import sympy as sp
from sympy import separatevars

# Display und Output
from IPython.display import display, Math, Latex, HTML

import textwrap

# Grafiken in EU-Größen ausgeben
def figsize_cm(width_cm, height_cm):
    return (2*width_cm / 2.54, 2*height_cm / 2.54)
def comma_to_float(valstr):
    return float(valstr.replace(',', '.'))

g=9.80984
Delta_g=0.00002

```

```

[2]: currentversuch = "243"
base = Path.cwd()
Messwerte_path = Path(base.parent.parent.parent/"Messwerte"/currentversuch)
    ↪ # initializing paths
Ausgabe_path = Path(base/"Diagramme")
print(Messwerte_path)
print(Ausgabe_path)

```

c:\Users\samue\OneDrive\Dokumente\PAP\PAP2\Messwerte\243

c:\Users\samue\OneDrive\Dokumente\PAP\PAP2\Auswertungen2.2\Code\243\Diagramme

Delete UTF8 False encoding

```

[3]: # def deletevocals(text):
#     text=textwrap.dedent(text)
#     patterna = r'["][a]'
#     text=re.sub(patterna,"ä",text)
#     patternu=r'["][u]'
#     text=re.sub(patternu,"ü",text)
#     patterno=r'["][o]'
#     text=re.sub(patterno,"ö",text)
#     return text

```

Regexpwissen: [0-9], [A-Z] findet egal welchen Zahl/Buchstaben; [^0-9] findet
↳ erstes Zeichen, das keine Zahl ist;

```
def deletevowels(text):
    # Einrückungen entfernen (gemeinsame Tabs des gesamten Textes)
    text = textwrap.dedent(text)
    # entfernt Bindestriche: - findet Bindestrich, \n Zeilenumbruch direkt
↳ dahinter, \s Whitespaces *beliebig viele
    text = re.sub(r'-\n\s*', "", text)
    # entfernt Zeilenumbrüche
    text = text.replace("\n", " ")

    # Eine Funktion, die vom re.sub aufgerufen wird, wenn ein Treffer erzielt
↳ wird
    def convert(match):
        # match.group(1) ist der Buchstabe nach dem Pünktchen (z.B. 'a' oder
↳ 'A')
        buchstabe = match.group(1)
        # Dictionary für die echten Umlaute
        umlaute = {'a': 'ä', 'u': 'ü', 'o': 'ö', 'A': 'Ä', 'U': 'Ü', 'O': 'Ö'}
        # Gib den Umlaut zurück. Falls das Zeichen nicht im Dict ist, lass es
↳ wie es ist
        return umlaute.get(buchstabe, buchstabe)
    # Das Pattern sucht nach ` gefolgt von einem beliebigen Buchstaben aus der
↳ Auswahl, [X] findet genau ein Zeichen aus Menge X
    return re.sub(r'`([auoAUO])', convert, text)
```

```
[67]: print(deletevowels("""
Die qualitativen Beobachtungen des Rauschens entsprachen im Grunde der
↳ theoretischen Erwartung:
Einerseits besagt die Nyquist-Beziehung gerade, dass die Effektivspannungen
↳ (das beobachtete Rau-
schen) mit erhöhten Widerständen steige
"""))
```

Die qualitativen Beobachtungen des Rauschens entsprachen im Grunde der
theoretischen Erwartung: Einerseits besagt die Nyquist-Beziehung gerade, dass
die Effektivspannungen (das beobachtete Rauschen) mit erhöhten Widerständen
steige

Runden signifikanter Stellen

```
[5]: def round_to_sigs(val, errVal, einheiten):
    """
    Funktion zur Rundung eines Fehlers und die Anpassung des Messwertes daran.
↳ Diese Funktion wurde etwas umständlicher
```

geschrieben, da python mit Floats und Runden schnell in Probleme rennt.
↳ Daher musste hier mit dezimal gearbeitet werden.

Zudem sollten besonders kleine und große Messwerte in der
↳ Dezimalschreibweise geschrieben werden, damit diese auch
für Protokolle geeignet sind.

Parameter

****val**** : float
Messwert

****errVal**** : float
Ungenauigkeit des Messwertes

Return

****value_rounded**** : str
Gerundeter Messwert

****error_rounded**** : str
Gerundete Ungenauigkeit des Messwertes

****res**** : str
"Messwert \pm Fehler"
"""

Daten zu Dezimal wechseln, da Floats probleme machen

val = Decimal(str(val))

errVal = Decimal(str(errVal))

exp = int(math.floor(math.log10(float(errVal)))) *# Die erste*
↳ *signifikante Stellenposition wird anhand des errVal bestimmt*

*if round(float(errVal / (Decimal(10) ** exp))) < 3:* *# wenn 1,2,3*
↳ *erste signifikante Stelle, dann kommt eine zweite Nachkommastellenposition*
↳ *hinzu*

exp -= 1

scale = Decimal(10) ** exp

val_round = (val / scale).quantize(Decimal('1'), rounding=ROUND_HALF_UP) *
↳ *scale* *# mit scale wird das Komma so verschoben, dass alle*
↳ *signifikanten Stellen vor dem Komma stehen, und dann alle Nachkommastellen*
↳ *weggerundet werden*

err_round = (errVal / scale).quantize(Decimal('1'), rounding=ROUND_CEILING)
↳ ** scale*

```

qty = f"\\qty{{{val_round}}({err_round})}}{{{einheiten}}}"
num = f"\\num{{{val_round}}({err_round})}}"
return float(val_round), float(err_round), num, qty

v_round = np.vectorize(round_to_sigs, otypes=[float, float, object, object],
↳excluded=['einheiten'])

# wissenschaftliche Notation erstmal vernachlässigen (da häufig Einheiten,
↳gewollt sind, die 10^/e Präfixe beinhalten)

```

Gaussche Fehlerfortpflanzung

```

[6]: def gff(function, errPronePar, latex=True):
    """
    Kann die Fehlerformel einer gegebenen Gleichung bestimmen.

    Parameters
    -----
    **func** : sympy function
        Funktion dessen Fehler bestimmt werden soll.

    **errPronePar** : Array
        Liste (Array) aller fehlerbehafteten Größen der Gleichung con sp.Symbols
        Diese Werte werden als x_sym, y_sym, z_sym etc. bezeichnet und sind,
    ↳ungleich den Werten für x, y, z.
        Für die Werte wird daher die Bezeichnung x_val, y_val, z_val etc.
    ↳genutzt und für deren Fehler err_x, err_y, err_z etc.

    Return
    -----
    **absolut_err** : sympy function
        Gibt die Fehlergleichung des absoluten Fehlers wieder.

    **relativ_err** : sympy function
        Gibt die Fehlergleichung des relativen Fehlers wieder.

    **errProneParameters** : array
        Liste aller Fehlerbehafteten Größen
    """

    error = 0
    errProneParameters = []

    function = sp.sympify(function)
    variables = errPronePar.split(",")
    variables = sp.symbols(variables)

```

```

for variable in variables:
    Delta = sp.Symbol(f"Delta_{variable}")
    partial = sp.diff(function, variable) # Die Funktion
    → wird nach der fehlerbehafteten Variable abgeleitet
    term=sp.UnevaluatedExpr(partial*Delta)**2
    error = error + ((term)) # Fehler werden
    → quadratisch aufsummiert
    errProneParamaters.append((variable,Delta))

absolute_err=sp.simplify(sp.sqrt(error),rational = True)
relative_err=sp.simplify(sp.sqrt(error/function**2),rational = True)

if latex:
    #Prints out the Latex Code for the Functions
    print("Funktion:")
    display(Math(sp.latex(function,long_frac_ratio=2)))
    print(sp.latex(function))
    print("Absoluter Fehler:")
    display(Math(sp.latex(absolute_err,long_frac_ratio=2)))
    print(sp.latex(absolute_err))
    print("Relativer Fehler:")
    display(Math(sp.latex(relative_err,long_frac_ratio=2)))
    print(sp.latex(relative_err))
    return function,absolute_err, relative_err, errProneParamaters

```

Sigma-Abweichungen

```

[7]: def sigma_abweichung(p1, err_p1,p2,err_p2):
    """
    Funktion zum berechnen der Sigma-Abweichugn von zwei Messwerten, oder einem
    → Messwert und einem Literaturwert.
    """
    if err_p1 == 0 and err_p2 == 0:
        raise ValueError("Für die Sigma-Abweichung muss mindestens ein Wert
        → fehlerbehaftet sein!")
    else:
        abweichung=Decimal(str(abs(p1 - p2)/(np.sqrt(err_p1 ** 2+err_p2 ** 2))))

    if abweichung == 0:
        return 0.0, "\\num{0}"

    exp = int(math.floor(math.log10(float(abweichung))))
    if round(float(abweichung / (Decimal(10) ** exp))) < 3: # wenn
        → 1,2,3 erste signifikante Stelle, dann kommt eine zweite
        → Nachkommastellenposition hinzu
        exp -= 1

```

```

scale = Decimal(10) ** exp
abweichung_round = (abweichung / scale).quantize(Decimal('1'),
↳rounding=ROUND_CEILING) * scale
res = f"\\num{{{abweichung_round}}}"

return float(abweichung_round),res

```

```

[8]: # #Größenvergleich in latex
# def size_comp_str(name1,name2,val1,val2):
#     if val1<val2:
#         return name1+"<"+name2
#     elif val1>val2:
#         return name1+">"+name2
#     else:
#         return name1+"="+name2

#Erstellung von Vergleichstabellen in Latex
def compare_table_array(nam1, nam2, einheiten, val1_array, err1_array,
↳val2_array, err2_array):
    # 1. Tabellenkopf (Hier werden die Strings einmalig eingesetzt)
    output_header = textwrap.dedent(f"""
        \\begin{{{table}}}[htb]
        \\centering
        \\caption{{Vergleich von ${nam1}$ und ${nam2}$}}
        \\begin{{{tabularx}}}{{0.8\\textwidth}}{{ZZZZZ}}
        \\toprule
        Messreihe & {{nam1}}[\\unit{{{einheiten}}}] &
↳{{nam2}}[\\unit{{{einheiten}}}] & \\text{{abs.Abw.}}[\\unit{{{einheiten}}}] &
↳\\text{{proz.Abw.}}[\\unit{{{percent}}}] & S[\\sigma] \\midrule
        """).strip() # strip macht whitespaces am anfang und ende des strings weg

    table_rows = []

    # 2. Der Body (Schleife läuft über die Werte-Arrays)
    for val1, err1, val2, err2 in zip(val1_array, err1_array, val2_array,
↳err2_array):

        # Deine Berechnungen (bleiben exakt gleich)
        VAL1 = round_to_sigs(val1, err1, r')[2]
        VAL2 = round_to_sigs(val2, err2, r')[2]

        abs_val = np.abs(val1 - val2)
        abs_delta = np.sqrt(err1**2 + err2**2)

        if abs_delta != 0:
            SIGMA = sigma_abweichung(val1, err1, val2, err2)[1]
        else:

```

```

        SIGMA = 0.0

        ABS = round_to_sigs(abs_val, abs_delta, r'')[2]

        rel = abs_val / np.abs(val1) * 100 if val1 != 0 else 0
        rel_delta = rel * np.sqrt((abs_delta / abs_val)**2 + (err1 / val1)**2)
    ↪if abs_val != 0 and val1 != 0 else 0
        REL = round_to_sigs(rel, rel_delta, r'')[2]

        # Eine saubere Zeile nur mit den Werten für LaTeX
        row = f"          & {VAL1} & {VAL2} & {ABS} & {REL} & {SIGMA} \\\\"
        table_rows.append(row)

    body_str = "\n".join(table_rows)

    # 3. Tabellenfuß
    output_footer = textwrap.dedent(f"""
        \\bottomrule
        \\end{{tabularx}}
        \\label{{table:Vergleich_{nam1}}}
        \\end{{table}}
    """).strip()

    # Alles zusammensetzen
    final_output = f"{output_header}\\n{body_str}\\n{output_footer}"

    print(final_output)

```

```

[ ]: # #Erstellung von Vergleichstabellen in Latex
# def compare_table_withliterature(nam1,nam2,einheiten,val1,err1,val2):
#     VAL1=round_to_sigs(val1,err1,r'')[2]
#     abs=np.abs(val1-val2)
#     abs_delta=np.sqrt(err1**2)
#     if abs_delta!=0:
#         SIGMA=sigma_abweichung(val1,err1,val2,0)[1]
#     else:
#         SIGMA=0.0
#     ABS=round_to_sigs(abs,abs_delta,r'')[2]
#     rel=abs/np.abs(val2)*100
#     rel_delta=rel*np.sqrt((abs_delta/abs)**2+(err1/val1)**2) if abs != 0 else
    ↪0
#     REL=round_to_sigs(rel,rel_delta,r'')[2]
#     output = textwrap.dedent(f"""
#         \\begin{{table}}[htb]
#         \\centering
#         \\caption{{}}
#         \\begin{{tabularx}}{{0.8\\textwidth}}{{ZZZZZ}}

```

```

#           \\toprule
#
#           □
#   ↪{nam1}[\\unit{{{einheiten}}}]&{nam2}[\\unit{{{einheiten}}}]&\\text{{abs.Abw.
#   ↪}}[\\unit{{{einheiten}}}]&\\text{{proz.Abw.
#   ↪}}[\\unit{{{percent}}}]&S[\\sigma]\\\\\\\\\\midrule
#           {VAL1}&\\num{{{val2}}}&{ABS}&{REL}&{SIGMA}\\\\\\\\
#           \\bottomrule
#           \\end{{tabularx}}
#           \\label{{table:Vergleich_{nam1}}}
#           \\end{{table}}
#   """)
#   print(output)

# #Größenvergleich in latex
# def size_comp_str(name1,name2,val1,val2):
#     if val1<val2:
#         return name1+"<"+name2
#     elif val1>val2:
#         return name1+">"+name2
#     else:
#         return name1+"="+name2

#Erstellung von Vergleichstabellen in Latex
import textwrap
def compare_table(nam1,nam2,einheiten,val1,err1,val2,err2):
    VAL1=round_to_sigs(val1,err1,r'[2]
    if err2!=0:
        VAL2=round_to_sigs(val2,err2,r'[2]
    else:
        VAL2=val2
    abs=np.abs(val1-val2)
    abs_delta=np.sqrt(err1**2+err2**2)
    if abs_delta!=0:
        SIGMA=sigma_abweichung(val1,err1,val2,err2)[1]
    else:
        SIGMA=0.0
    ABS=round_to_sigs(abs,abs_delta,r'[2]
    rel=abs/np.abs(val1)*100
    rel_delta=rel*np.sqrt((abs_delta/abs)**2+(err1/val1)**2) if abs != 0 else 0
    REL=round_to_sigs(rel,rel_delta,r'[2]
    output = textwrap.dedent(f"""
        \\begin{{table}}[htb]
        \\centering
        \\caption{{}}
        \\begin{{tabularx}}{{\\textwidth}}{{ZZZZx}}

```

I Zeitkonstanten

Funktion:

C R

Absoluter Fehler:

$$\sqrt{C^2 \Delta_R^2 + \Delta_C^2 R^2}$$

$$\sqrt{C^2 \Delta_R^2 + \Delta_C^2 R^2}$$

Relativer Fehler:

$$\sqrt{\frac{\Delta_R^2}{R^2} + \frac{\Delta_C^2}{C^2}}$$

$$\sqrt{\frac{\Delta_R^2}{R^2} + \frac{\Delta_C^2}{C^2}}$$

```
[11]: (C*R,
      sqrt(C**2*Delta_R**2 + Delta_C**2*R**2),
      sqrt(Delta_R**2/R**2 + Delta_C**2/C**2),
      [(R, Delta_R), (C, Delta_C)])
```

```
[12]: gff("T_12/ln(2)", "T_12")
```

Funktion:

$$\frac{T_{12}}{\log(2)}$$

$$\frac{T_{12}}{\log(2)}$$

Absoluter Fehler:

$$\frac{\sqrt{\Delta_{T_{12}}^2}}{\log(2)}$$

$$\frac{\sqrt{\Delta_{T_{12}}^2}}{\log(2)}$$

Relativer Fehler:

$$\sqrt{\frac{\Delta_{T_{12}}^2}{T_{12}^2}}$$

$$\sqrt{\frac{\Delta_{T_{12}}^2}{T_{12}^2}}$$

```
[12]: (T_12/log(2),
      sqrt(Delta_T_12**2)/log(2),
      sqrt(Delta_T_12**2/T_12**2),
      [(T_12, Delta_T_12)])
```

II Differnetation einer Rechteckspannung

Vorversuch Rauschspektrum

```
[13]: def load_and_save_clean_csv(file_path, skiprowsnumber):
      # 1. Den vollen Pfad zur Originaldatei erstellen
      original_path = Messwerte_path / file_path

      # 2. Die kaputte Datei einlesen
```

```

with open(original_path, "r", encoding="utf-8") as f:
    text = f.read()

    # 3. Den Text bereinigen (Tausender-Kommas entfernen)
    # macht großteil richtig: ^ Sucht nur am anfang einer Zeile; (\d+) sucht
    ↪erste Zahl 124 bei 124,111.000;
    # , sucht Komma direkt hinter Zahlenblock; (\d+) sucht nächsten Zahlenblock
    ↪111; \. maskierter Punkt (. wäre jegliches Zeichen)
    # r"\1\2. entfernt das Komma zwischen den Gruppen
    sauberer_text = re.sub(r"^(\d+),(\d+)\.", r"\1\2.", text, flags=re.
    ↪MULTILINE)

    # fixt kleine ausnahmen, die keine Nachkommastellen haben, weil exakt .0:
    ↪15,625 wurde nicht erkannt und blieb 15,625.
    # ,(\d{3}) erkennt genau die hundertergruppe, die immer 3 Ziffern hat
    sauberer_text = re.sub(r"^(\d+),(\d{3})", r"\1\2.", sauberer_text,
    ↪flags=re.MULTILINE)

    # jetzt blieben noch 1(,)111,111.111 Zahlen, also Megahertz freq, deren
    ↪erstes Komma wurde in Iteration 1 schon entfernt. Jetzt fehlt also noch das
    ↪zweite.
    sauberer_text = re.sub(r"^(\d+),(\d{3})", r"\1\2.", sauberer_text, flags=re.
    ↪MULTILINE)

    # 4. Den neuen Dateipfad im SELBEN Ordner generieren
    # original_path.parent ist der Ordner (Messwerte_path)
    # original_path.name ist z.B. 'deine_datei.csv'
    neu_datei_path = original_path.parent / f"bereinigt_{original_path.name}" #
    ↪fixt den bug für zahlen >10^6, bei denen ., von einer vorherigen sub
    ↪geschrieben wurde

    # 5. Den bereinigten Text in die neue Datei schreiben
    with open(neu_datei_path, "w", encoding="utf-8") as f:
        f.write(sauberer_text)

    print(f"Saubere Datei wurde erstellt unter: {neu_datei_path}")

    # 6. Die Daten nun ganz normal aus der neu erstellten Datei laden
    data = np.loadtxt(neu_datei_path, delimiter=",", skiprows=skiprowsnumber,
    ↪usecols=(0, 1), unpack=True)

    return data

# --- Anwendung ---
# frequenz, amplitude = load_and_save_clean_csv('deine_datei.csv')

```

```
[14]: plt.close('all')
plt.figure(figsize=(figsize_cm(10.8,6)))
plt.title(r'Rauschspektrum für $f=100$ Hz bis 2 MHz')
plt.xlabel(r'$f$ [Hz]')
plt.ylabel(r'Pegel [dBm]')

f_5,P_5 = load_and_save_clean_csv('5kohm_fspectrum.csv',32)
f_30,P_30 = load_and_save_clean_csv('30kohm_fspectrum.csv',32)

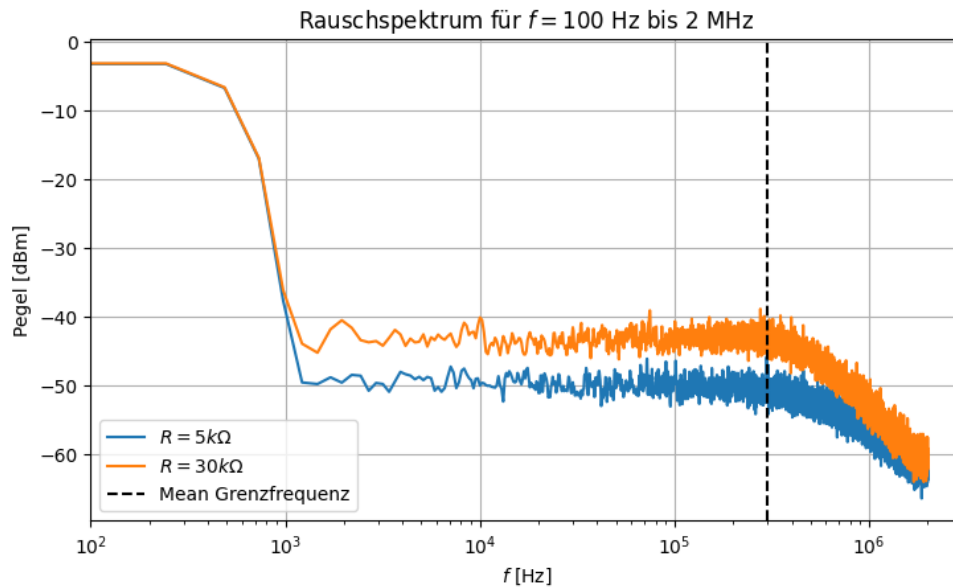
plt.plot(f_5,P_5,label=rf'$R=5k\Omega$')
plt.plot(f_30,P_30,label=rf'$R=30k\Omega$')
plt.axvline(f_5[1230],linestyle='--',color='black',label='Mean Grenzfrequenz')
diff=np.mean(P_5[:1230]-P_30[:1230])
print(diff)
print(f_5[1230])
plt.xlim(100,3e6)
plt.xscale('log')
plt.legend()
plt.grid()
# plt.tight_layout()
plt.savefig(Ausgabe_path/"Rauschpegel_5_30.
    ↪png",format="png",bbox_inches='tight')
plt.show()
```

Saubere Datei wurde erstellt unter: c:\Users\samue\OneDrive\Dokumente\PAP\PAP2\Messwerte\243\bereinigt_5kohm_fspectrum.csv

Saubere Datei wurde erstellt unter: c:\Users\samue\OneDrive\Dokumente\PAP\PAP2\Messwerte\243\bereinigt_30kohm_fspectrum.csv

-7.0924460479928895

300292.96875



I Frequenzgang

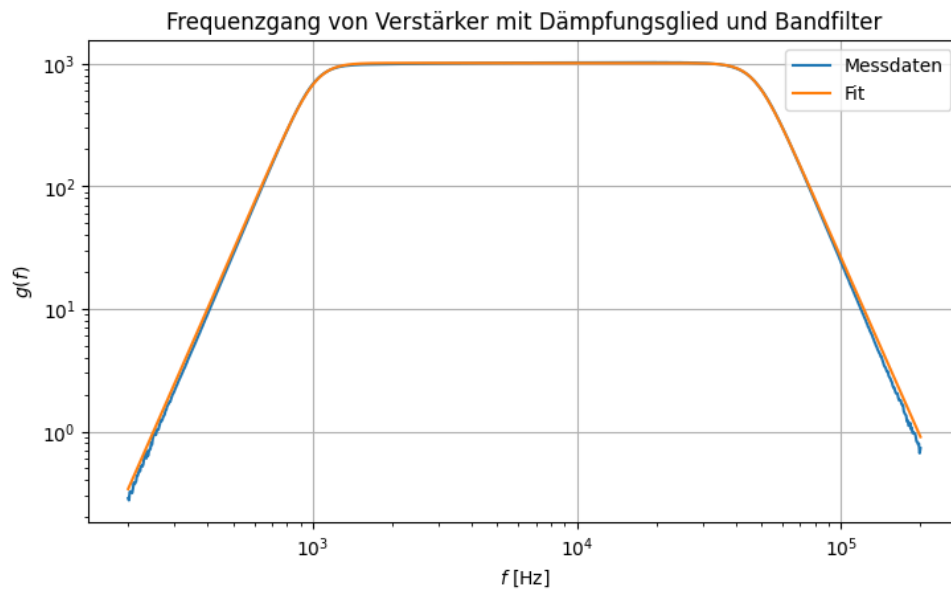
```
[ ]: plt.close('all')
plt.figure(figsize=(figsize_cm(10.8,6)))
plt.title(r'Frequenzgang von Verstärker mit Dämpfungsglied und Bandfilter')
plt.xlabel(r'$f$ [Hz]')
plt.ylabel(r'$g(f)$')

f,P = load_and_save_clean_csv('frequenzgang.csv',17)
D=1e-3
g=P/D
plt.loglog(f,g,label=rf'Messdaten')
# plt.xlim(100,3e6)
# plt.xscale('log')
# plt.yscale('log')
def fit_func(f,V,W1,W2,n1,n2):
    return V/(np.sqrt(1+1/(f/W1)**(2*n1))*np.sqrt(1+(f/W2)**(2*n2)))
p0 = [1000 , 1000 , 50000 , 5 , 5]
popt, pcov = curve_fit(fit_func, f, g ,p0)
plt.loglog(f, fit_func(f, *popt), label='Fit')
for i in range(len(p0)):
    wert,fehler,latex,_=round_to_sigs(popt[i],np.sqrt(pcov[i][i]),einheiten='')
    print(latex)
    print(fehler/wert*100)
plt.legend()
plt.grid()
# plt.tight_layout()
```

```
plt.savefig(Ausgabe_path/"Frequenzgang_Fit.
↳png",format="png",bbox_inches='tight')
plt.show()
```

Saubere Datei wurde erstellt unter: c:\Users\samue\OneDrive\Dokumente\PAP\PAP2\Messwerte\243\bereinigt_frequenzgang.csv

```
\num{1010.1(0.4)}
0.039600039600039605
\num{1024.2(0.8)}
0.07810974419058778
\num{47190(40)}
0.0847637211273575
\num{4.896(0.019)}
0.3880718954248366
\num{4.861(0.019)}
0.39086607693890146
```



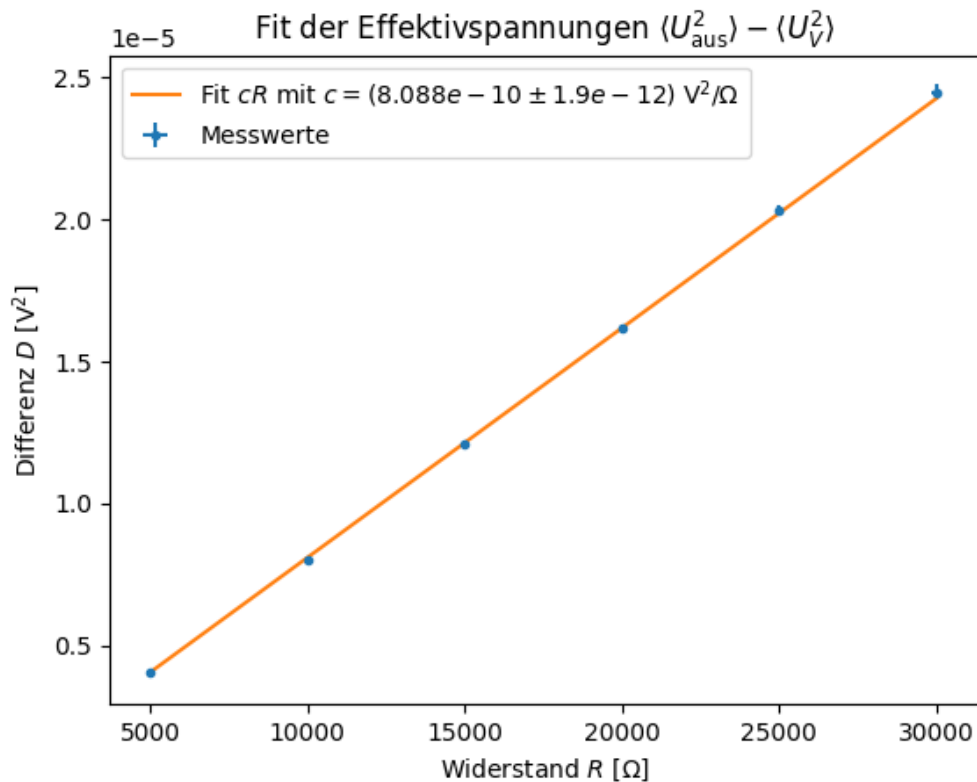
```
[ ]: def fit_func_square(f,V,W1,W2,n1,n2):
    return fit_func(f,V,W1,W2,n1,n2)**2
B=integrate.quad(fit_func_square, f[0], f[-1], args=tuple(popt))
B,Delta_B,_,latexB=round_to_sigs(B[0],B[0]*0.02,r'\hertz')
print(latexB)
# print(f'Das Integral beträgt: {round_to_sigs(B,B*0.02,r'\hertz')[2]}')
```

```
\qty{47900000000(1000000000)}{\hertz}
```

```
[65]: R=np.array([5e3, 10e3 , 15e3 , 20e3 , 25e3 , 30e3]) #Eingabe der Widerstände
Delta_R=R*0.005
U_aus=np.array([2.4436,3.1588,3.7474,4.2562,4.7182,5.1399])*1e-3
Delta_U_aus=np.array([0.0098,0.0132,0.0143,0.0158,0.0184,0.027])*1e-3
U_V=1.3958*1e-3
Delta_U_V=0.0116*1e-3
D=U_aus**2-U_V**2 #Berechnung Differenz
Delta_D=np.sqrt((2*Delta_U_aus*U_aus)**2 + (2*Delta_U_V*U_V)**2)

plt.close('all')
plt.errorbar(R, D, yerr=Delta_D, xerr= Delta_R, fmt='.',label='Messwerte')
# plt.axis([0,3.5e4,0,2.5e-5])
def linear(x,c): #Definition einer linearen Funktion als Fit
    return c*x
popt1, pcov1 = curve_fit(linear, R, D , sigma=Delta_D) #Durchführung des Fits
c,Delta_c,_,latexc=round_to_sigs(popt1[0],np.
    ↪sqrt(pcov1[0][0]),r'\Volt\squared\per\Omega')
plt.plot(R,linear(R,*popt1),label = rf'Fit $cR$ mit $c=({c}\pm{Delta_c})\backslash$
    ↪\mathrm{{V}}^2/\Omega$')
print(latexc)
plt.title(r'Fit der Effektivspannungen $\angle U_{\text{aus}}^2\angle-\angle U_{\text{V}}^2\angle$')
plt.xlabel(r'Widerstand $R$ $\backslash\Omega$')
plt.ylabel(r'Differenz $D$ $\backslash\text{V}^2$')
plt.legend()
plt.savefig(Ausgabe_path/"Differenz_D.png",format="png",bbox_inches='tight')
plt.show()
```

$\backslash\text{qty}\{8.088\text{E}-10(1.9\text{E}-12)\}\backslash\text{Volt}\backslash\text{squared}\backslash\text{per}\backslash\Omega\}$



```
[ ]: chisquare=np.sum(((linear(R,*popt1)-D)**2/Delta_D**2))
dof=5 #degrees of freedom
chisquare_red=chisquare/dof
print(chisquare)
print(chisquare_red)

from scipy.stats import chi2
prob=round(1-chi2.cdf(chisquare,dof),4)*100
print("Wahrscheinlichkeit="+str(prob)+"%")
```

1.4947555699945816

0.29895111399891633

Wahrscheinlichkeit=91.36999999999999%

```
[ ]: T = 25.1 + 273.15
Delta_T = 0.5
k = c/(4*T*B)
Delta_k_syst = k * Delta_B/B
Delta_k_stat = k*np.sqrt(Delta_T**2/T**2 + Delta_c**2/c**2)
_,Delta_k_syst,_,latexk=round_to_sigs(k,Delta_k_syst,'')
k,Delta_k_stat,_,_=round_to_sigs(k,Delta_k_stat,r'\joule\per\kelvin')
```

```
print('k=',k, Delta_k_stat, Delta_k_syst)
```

```
k= 1.415e-23 5e-26 3e-25
0.021201413427561835
```

```
[64]: compare_table(r'k_{\text{exp}}',r'k_{\text{lit}}',r'\joule\per\kelvin',1.415,0.03,1.
↪380649,0)
```

```
\begin{table}[htb]
\centering
\caption{}
\begin{tabularx}{0.8\textwidth}{ZZZZZ}
\toprule
k_{\text{exp}}[\unit{\joule\per\kelvin}]&k_{\text{lit}}[\unit{\joule\per\kelvin}]&\text{abs.Abw.}[\unit{\joule\per\kelvin}]&\text{proz.Abw.}[\unit{\percent}]&S[\sigma]\\\midrule
\text{1.42(0.03)}&\text{1.380649}&\text{0.03(0.03)}&\text{2.4(2.2)}&\text{1.2}\\\bottomrule
\end{tabularx}
\label{table:Vergleich_k_{\text{exp}}}
\end{table}
```

```
[51]: gff("c/(4*T*B)", "c,T")
```

Funktion:

$$\frac{c}{4BT}$$

$\frac{c}{4BT}$

Absoluter Fehler:

$$\frac{1}{4} \sqrt{\frac{1}{B^2 T^4} (\Delta_T^2 c^2 + \Delta_c^2 T^2)}$$

$$\frac{\sqrt{\frac{\Delta_T^2}{B^2 T^4} c^2 + \frac{\Delta_c^2}{T^4}}}{4}$$

Relativer Fehler:

$$\sqrt{\frac{\Delta_T^2}{T^2} + \frac{\Delta_c^2}{c^2}}$$

$$\sqrt{\frac{\Delta_T^2}{T^2} + \frac{\Delta_c^2}{c^2}}$$

```
[51]: (c/(4*B*T),
sqrt((Delta_T**2*c**2 + Delta_c**2*T**2)/(B**2*T**4))/4,
sqrt(Delta_T**2/T**2 + Delta_c**2/c**2),
[(c, Delta_c), (T, Delta_T)])
```